

ZERO TO JOB-READY DATA ENGINEER 2026

M15 - Azure + Microsoft Fabric Data Engineering

Beginner-First Professional Institute Edition

This book is your teacher

16 lessons | live cloud/UI evidence separated from offline design | current Microsoft Learn authority

How to study M15

Teacher route

Understand -> assigned visual/current Microsoft Learn -> follow once -> close source -> CREATE your own attempt -> validate actual state -> teach back -> protected Review only after saved attempt -> changed retry if weak -> evidence/mastery.

Cloud/UI evidence rule

A portal screenshot from documentation is not your evidence. LIVE FABRIC means you actually created/opened/executed the named item and saved observed item/run/path/permission/data controls. If tenant/capacity/access is unavailable, label the work OFFLINE PLAN and do not relabel it as live later without executing it.

Runtime correction - September 2026

abric Runtime 1.3 as the latest GA/default runtime: Spark 3.5.5, Python 3.11, Java 11, Delta 3.2.
Spark 4.1, Python 3.13, Java 21, Delta 4.2. The open-source M12/M13 versions differ intentionally.

No percentage shortcut

M15 completion requires every required competency with genuine evidence and zero unresolved critical failures. Missing live Fabric proof remains runtime/UI pending; an average cannot hide it.

Contents / Index

DE15-01	Cloud fundamentals for data engineers	6
DE15-02	Identity, IAM and least privilege	9
DE15-03	Azure Storage concepts and OneLake	12
DE15-04	Fabric workspace and item model	15
DE15-05	OneLake paths, files, tables and shortcuts	18
DE15-06	Fabric Lakehouse and Delta tables	21
DE15-07	Fabric notebooks	24
DE15-08	Spark in Fabric	27
DE15-09	Fabric Data Pipelines	30
DE15-10	SQL analytics endpoint and warehouse choice	33
DE15-11	Medallion architecture in Fabric	36
DE15-12	Incremental loads and MERGE in cloud	39
DE15-13	Permissions, connections and secrets	42
DE15-14	Monitoring, reliability and cost awareness	45
DE15-15	Git, deployment and environment separation	48
DE15-16	End-to-end Fabric data engineering case	51

Closing pages

p54 Mastery checklist | p55 Revision quick reference | p56 M16/G06 route.

Module map & first-use access

DE15-01-05	Cloud mental model -> identity -> OneLake -> workspace/items -> paths/shortcuts
DE15-06-10	Lakehouse/Delta -> notebooks -> Fabric Spark runtime -> pipelines/Copy job -> SQL endpoint/warehouse choice
DE15-11-13	Medallion -> incremental/MERGE -> permissions/connections/secrets
DE15-14-16	Monitoring/cost -> Git/deployment -> end-to-end evidence case
Next	M16 then M17; G06 stays locked until M15-M17 evidence is complete

Install/access only here

M15 is the first Fabric module. Use a browser and your authorized Fabric tenant/trial/capacity if available. Do not install Azure CLI, PowerShell modules or other local tooling just to imitate a portal lesson. Record the tenant/workspace/runtime actually visible to you.

Runtime choice

For ordinary ☐ start from the current GA runtime unless your actual workspace intentionally uses a preview. If you choose ☐ mark it PREVIEW in evidence and expect behavior/library changes.

Source, workspace & evidence policy

Every lesson preserves its visual route and current Microsoft Learn authority as clickable links. Product behavior is controlled by current Microsoft Learn, not by an old screenshot. After viewing, close the source before independent work.

LIVE FABRIC	Your actual item/path/run/permission/data observation.
OFFLINE PLAN	Architecture/security/run design that has NOT executed in Fabric.
SIMULATOR	Local deterministic data-control output; NOT Fabric runtime.
FAILED ATTEMPT	Preserved append-only evidence used to drive targeted repair.
CHANGED RETRY	Fresh proof after Review; never overwrite first attempt.

DE15-01 - Cloud fundamentals for data engineers

1. Why this matters

Cloud data engineering replaces “one powerful laptop” with managed identity, storage, compute, orchestration and monitoring services. The engineering problem stays the same: know the input grain, preserve raw evidence, transform reproducibly, validate outputs and recover safely. Fabric is Microsoft’s integrated analytics platform; OneLake is its organization-wide logical lake. Cloud convenience does not remove the need to reason about state, cost, permissions or failure. In a job, a wrong assumption here can create data loss, excess privilege, unreproducible runs, or a cloud bill with no trustworthy output.

2. What you are learning

You will explain and apply Cloud fundamentals for data engineers, then produce this proof: one architecture map that labels source, persistent state, compute/orchestration, consumer, identity boundary and monitoring signal.

3. Prerequisite / start state

G05 passed and M14 complete. Start with a blank lesson workspace; do not begin from the Review or a finished architecture plan.

4. Picture it in your head

Think of cloud engineering as renting a secure industrial campus: OneLake/lakehouse is durable storage, notebooks/Spark are machines, pipelines are supervisors, identities are badges, and monitoring is the control room. Machines can stop; durable state and evidence must survive.

5. New words before we use them

managed service	Cloud capability operated by the provider rather than your laptop.
durable state	Data/configuration that must survive compute stopping.
orchestration	Coordinating dependent processing steps.
capacity	Allocated cloud compute resources/cost boundary.

DE15-01 - Visual + follow with me once

6. Useful video / current source route

[VIDEO FIRST: Microsoft Learn DP-700 — Implement/manage \(Part 1\)](#)

Use: 03:07 onward — analytics-solution responsibilities and Fabric architecture context | Status: PRESERVED_VISUAL_ROUTE_WITH_MICROSOFT_LEARN_FALLBACK_2026-09-11

[CURRENT OFFICIAL REFERENCE: DP-700T00-A course](#)

Use: Current product behavior and exact lesson reference | Status: CURRENT_MICROSOFT_LEARN_2026-09-11

7. Follow with me once

Use the normal Retail Orders fixture or a tiny named Fabric object. Before clicking anything, write the expected object/state and whether this will be LIVE FABRIC or OFFLINE PLAN. 1. Open the normal learner_architecture_plan.json and leave every field blank first. 2. List API/files as sources and name one durable raw state in OneLake/lakehouse. 3. Add notebook/pipeline as compute/control flow and SQL/BI as consumer surfaces. 4. Add one identity boundary and one monitoring signal; then compare with EXPECTED proof, not a finished answer. After each step, record the exact item/path/identity/run evidence you actually observed. If the portal or tenant is unavailable, stop the live claim, keep the design labeled OFFLINE, and use the official reference only to understand the object—not to fabricate a successful run.

```
Source -> Bronze durable state -> Notebook/Spark -> Silver/Gold -> SQL/BI
      | identity | pipeline | monitoring |
```

8. What you should see / be able to say

one architecture map that labels source, persistent state, compute/orchestration, consumer, identity boundary and monitoring signal.

9. Why it worked

The exercise separates the Fabric object from the durable state it controls, then attaches identity, validation and failure evidence. That makes the result explainable even when the portal layout changes.

DE15-01 - Solo proof, repair and mastery

10. Common beginner mistakes

- Copying portal steps without understanding the object/state created.
- Assuming an Admin experience equals a least-privilege user experience.
- Treating a successful run badge as enough without data reconciliation.
- Hard-coding workspace IDs, paths, credentials or environment-specific values.

11. If you get stuck

Preserve the first symptom. Check one boundary at a time: tenant/capacity -> identity/permission -> workspace/item -> path/grain/schema -> notebook/pipeline state -> serving -> monitoring/lifecycle. Compare current Microsoft Learn before changing multiple settings. If access is unavailable, stop and mark LIVE FABRIC pending.

12. You do a small version alone

Design a new Fabric architecture for a small parcel company with API + files + analytics consumers. Name state, compute, security and monitoring boundaries.

13. Validate

one architecture map that labels source, persistent state, compute/orchestration, consumer, identity boundary and monitoring signal. Label every proof LIVE FABRIC, OFFLINE PLAN or SIMULATOR. A green status alone is never sufficient.

14. Teach it back

Explain cloud fundamentals for data engineers for 60–90 seconds. Name the object, the state it controls, one failure mode and one validation.

15. Protected review + changed retry

Save 04_PRACTICE_FILES/lesson_workspaces/DE15-01/attempt.md first. Only then open the protected Review. If weak, close Review and do this changed retry separately: Design a changed Fabric architecture for a small parcel company with API + files + analytics consumers. Name state, compute, security and monitoring boundaries. Preserve the failed attempt append-only.

16. Evidence / mastery + exact next route

CREATE evidence/ only for genuine observed proof. Mastery requires the independent/changed proof, validation, teach-back, and no unresolved critical issue. Exact next route: DE15-02.

DE15-02 - Identity, IAM and least privilege

1. Why this matters

Identity answers who/what is calling; authentication proves that identity; authorization decides what it may do. Least privilege means grant only the minimum actions and data scope needed. In Fabric, workspace roles are control-plane permissions while OneLake security provides data-plane controls. A Contributor can create/modify many workspace items, so do not hand out workspace roles just to grant narrow data read access. In a job, a wrong assumption here can create data loss, excess privilege, unreproducible runs, or a cloud bill with no trustworthy output.

2. What you are learning

You will explain and apply Identity, IAM and least privilege, then produce this proof: a permission matrix with at least one deliberately refused grant and one test performed as the intended narrow identity.

3. Prerequisite / start state

DE15-01 mastered. Start with a blank lesson workspace; do not begin from the Review or a finished architecture plan.

4. Picture it in your head

Picture three doors: identity says whose badge this is, authentication verifies the badge, authorization decides which doors open. Workspace roles are broad building access; OneLake roles can narrow access to data.

5. New words before we use them

identity	User, group, service principal or workload identity.
authentication	Proof of who/what the caller is.
authorization	Decision about allowed actions/resources.
least privilege	Minimum access required for the job.

DE15-02 - Visual + follow with me once

6. Useful video / current source route

[VIDEO FIRST: Microsoft Learn DP-700 — Implement/manage \(Part 1\)](#)

Use: Use the Implement/manage security/permissions portion; exact current security rules come from the linked OneLake docs | Status:

PRESERVED_VISUAL_ROUTE_WITH_MICROSOFT_LEARN_FALLBACK_2026-09-11

[CURRENT OFFICIAL REFERENCE: OneLake security access-control model](#)

Use: Current product behavior and exact lesson reference | Status: CURRENT_MICROSOFT_LEARN_2026-09-11

7. Follow with me once

Use the normal Retail Orders fixture or a tiny named Fabric object. Before clicking anything, write the expected object/state and whether this will be LIVE FABRIC or OFFLINE PLAN. 1. Create three rows: analyst, engineer, pipeline service identity. 2. For each, write only the minimum job it must perform. 3. Choose narrow workspace/item/data rights; write one tempting but excessive grant and refuse it. 4. Write how you would test the analyst and service identity separately. After each step, record the exact item/path/identity/run evidence you actually observed. If the portal or tenant is unavailable, stop the live claim, keep the design labeled OFFLINE, and use the official reference only to understand the object—not to fabricate a successful run.

Identity	Job needed	Minimum scope	Explicitly denied
Analyst	Read Gold	narrow read	write/create
Engineer	Build	dev items	tenant admin
Pipeline	read/write	source+target	broad workspace

8. What you should see / be able to say

a permission matrix with at least one deliberately refused grant and one test performed as the intended narrow identity.

9. Why it worked

The exercise separates the Fabric object from the durable state it controls, then attaches identity, validation and failure evidence. That makes the result explainable even when the portal layout changes.

DE15-02 - Solo proof, repair and mastery

10. Common beginner mistakes

- Copying portal steps without understanding the object/state created.
- Assuming an Admin experience equals a least-privilege user experience.
- Treating a successful run badge as enough without data reconciliation.
- Hard-coding workspace IDs, paths, credentials or environment-specific values.

11. If you get stuck

Preserve the first symptom. Check one boundary at a time: tenant/capacity -> identity/permission -> workspace/item -> path/grain/schema -> notebook/pipeline state -> serving -> monitoring/lifecycle. Compare current Microsoft Learn before changing multiple settings. If access is unavailable, stop and mark LIVE FABRIC pending.

12. You do a small version alone

Create a least-privilege permission matrix for a Viewer analyst, Contributor engineer and service identity; defend every grant.

13. Validate

a permission matrix with at least one deliberately refused grant and one test performed as the intended narrow identity. Label every proof LIVE FABRIC, OFFLINE PLAN or SIMULATOR. A green status alone is never sufficient.

14. Teach it back

Explain identity, iam and least privilege for 60–90 seconds. Name the object, the state it controls, one failure mode and one validation.

15. Protected review + changed retry

Save 04_PRACTICE_FILES/lesson_workspaces/DE15-02/attempt.md first. Only then open the protected Review. If weak, close Review and do this changed retry separately: Create a least-privilege permission matrix for a Viewer analyst, Contributor engineer and service identity; defend every grant. Preserve the failed attempt append-only.

16. Evidence / mastery + exact next route

CREATE evidence/ only for genuine observed proof. Mastery requires the independent/changed proof, validation, teach-back, and no unresolved critical issue. Exact next route: DE15-03.

DE15-03 - Azure Storage concepts and OneLake

1. Why this matters

Azure storage concepts help explain Fabric: durable object/file storage is separated from compute. OneLake gives the tenant one logical lake namespace and can reference external data through shortcuts instead of copying it. The engineering choice is copy vs reference: copy when you need an independent governed snapshot/transform path; shortcut when live governed access without duplication is appropriate. In a job, a wrong assumption here can create data loss, excess privilege, unreproducible runs, or a cloud bill with no trustworthy output.

2. What you are learning

You will explain and apply Azure Storage concepts and OneLake, then produce this proof: three copy-vs-shortcut decisions with freshness, ownership and replay/recovery reasons.

3. Prerequisite / start state

DE15-02 mastered. Start with a blank lesson workspace; do not begin from the Review or a finished architecture plan.

4. Picture it in your head

Picture OneLake as one organization-wide address space. A copy creates bytes you own; a shortcut is a governed pointer to bytes elsewhere. Freshness, ownership and replay determine which is safer.

5. New words before we use them

OneLake	Fabric tenant-wide logical data lake.
shortcut	Pointer exposing external/other OneLake data without copying bytes.
copy	Physical duplication you own/manage.
namespace	Organized naming/address space for data.

DE15-03 - Visual + follow with me once

6. Useful video / current source route

[VIDEO FIRST: Microsoft Learn DP-700 — Implement/manage \(Part 1\)](#)

Use: 03:46 onward — workspace/storage context; then current OneLake reference | Status: PRESERVED_VISUAL_ROUTE_WITH_MICROSOFT_LEARN_FALLBACK_2026-09-11

[CURRENT OFFICIAL REFERENCE: OneLake documentation](#)

Use: Current product behavior and exact lesson reference | Status: CURRENT_MICROSOFT_LEARN_2026-09-11

7. Follow with me once

Use the normal Retail Orders fixture or a tiny named Fabric object. Before clicking anything, write the expected object/state and whether this will be LIVE FABRIC or OFFLINE PLAN. 1. Take external Parquet, daily CSV and shared Delta source. 2. For each, write freshness, ownership and replay/audit needs. 3. Choose COPY or SHORTCUT from those requirements, not convenience. 4. Write one failure/recovery consequence for each choice. After each step, record the exact item/path/identity/run evidence you actually observed. If the portal or tenant is unavailable, stop the live claim, keep the design labeled OFFLINE, and use the official reference only to understand the object—not to fabricate a successful run.

Decision = freshness + ownership + replay/audit
Need independent daily replay snapshot? -> COPY
Need governed live reference, no duplicate bytes? -> SHORTCUT

8. What you should see / be able to say

three copy-vs-shortcut decisions with freshness, ownership and replay/recovery reasons.

9. Why it worked

The exercise separates the Fabric object from the durable state it controls, then attaches identity, validation and failure evidence. That makes the result explainable even when the portal layout changes.

DE15-03 - Solo proof, repair and mastery

10. Common beginner mistakes

- Copying portal steps without understanding the object/state created.
- Assuming an Admin experience equals a least-privilege user experience.
- Treating a successful run badge as enough without data reconciliation.
- Hard-coding workspace IDs, paths, credentials or environment-specific values.

11. If you get stuck

Preserve the first symptom. Check one boundary at a time: tenant/capacity -> identity/permission -> workspace/item -> path/grain/schema -> notebook/pipeline state -> serving -> monitoring/lifecycle. Compare current Microsoft Learn before changing multiple settings. If access is unavailable, stop and mark LIVE FABRIC pending.

12. You do a small version alone

Choose copy or shortcut for a changed source set and record one failure/recovery concern per choice.

13. Validate

three copy-vs-shortcut decisions with freshness, ownership and replay/recovery reasons. Label every proof LIVE FABRIC, OFFLINE PLAN or SIMULATOR. A green status alone is never sufficient.

14. Teach it back

Explain azure storage concepts and onelake for 60–90 seconds. Name the object, the state it controls, one failure mode and one validation.

15. Protected review + changed retry

Save 04_PRACTICE_FILES/lesson_workspaces/DE15-03/attempt.md first. Only then open the protected Review. If weak, close Review and do this changed retry separately: Choose copy or shortcut for a changed source set and record one failure/recovery concern per choice. Preserve the failed attempt append-only.

16. Evidence / mastery + exact next route

CREATE evidence/ only for genuine observed proof. Mastery requires the independent/changed proof, validation, teach-back, and no unresolved critical issue. Exact next route: DE15-04.

DE15-04 - Fabric workspace and item model

1. Why this matters

A Fabric workspace is a collaboration/security boundary containing items such as lakehouses, notebooks, pipelines, warehouses and semantic models. Admin/Member/Contributor/Viewer are not “seniority levels”; they are permission bundles. Item type matters because each has a different execution/state model. Name workspaces and items so dev/test/prod intent is visible. In a job, a wrong assumption here can create data loss, excess privilege, unreproducible runs, or a cloud bill with no trustworthy output.

2. What you are learning

You will explain and apply Fabric workspace and item model, then produce this proof: a workspace/item inventory naming purpose, owner, input/output and validation evidence for every item.

3. Prerequisite / start state

DE15-03 mastered. Start with a blank lesson workspace; do not begin from the Review or a finished architecture plan.

4. Picture it in your head

A workspace is the project room; items are specialized machines inside it. A lakehouse stores, a notebook computes, a pipeline orchestrates, and a warehouse/SQL endpoint serves different query needs.

5. New words before we use them

workspace	Fabric collaboration/security container for items.
item	Fabric object such as lakehouse, notebook, pipeline or warehouse.
role	Permission bundle such as Admin/Member/Contributor/Viewer.
environment	Dev/test/prod execution context.

DE15-04 - Visual + follow with me once

6. Useful video / current source route

[VIDEO FIRST: Microsoft Learn DP-700 — Implement/manage \(Part 1\)](#)

Use: 03:46 onward — workspace configuration and item context | Status: PRESERVED_VISUAL_ROUTE_WITH_MICROSOFT_LEARN_FALLBACK_2026-09-11

[CURRENT OFFICIAL REFERENCE: Fabric workspace roles](#)

Use: Current product behavior and exact lesson reference | Status: CURRENT_MICROSOFT_LEARN_2026-09-11

7. Follow with me once

Use the normal Retail Orders fixture or a tiny named Fabric object. Before clicking anything, write the expected object/state and whether this will be LIVE FABRIC or OFFLINE PLAN. 1. Create a dev workspace name that exposes environment and purpose. 2. Add lakehouse, notebook, pipeline and serving item one at a time. 3. For each item write input, output/state, owner and proof-of-success. 4. Check that no two items have the same vague responsibility. After each step, record the exact item/path/identity/run evidence you actually observed. If the portal or tenant is unavailable, stop the live claim, keep the design labeled OFFLINE, and use the official reference only to understand the object—not to fabricate a successful run.

```
de-retail-dev
  retail_lh      -> durable data
  nb_silver_orders -> transform
  pl_orders      -> orchestration
  SQL_endpoint   -> read serving
```

8. What you should see / be able to say

a workspace/item inventory naming purpose, owner, input/output and validation evidence for every item.

9. Why it worked

The exercise separates the Fabric object from the durable state it controls, then attaches identity, validation and failure evidence. That makes the result explainable even when the portal layout changes.

DE15-04 - Solo proof, repair and mastery

10. Common beginner mistakes

- Copying portal steps without understanding the object/state created.
- Assuming an Admin experience equals a least-privilege user experience.
- Treating a successful run badge as enough without data reconciliation.
- Hard-coding workspace IDs, paths, credentials or environment-specific values.

11. If you get stuck

Preserve the first symptom. Check one boundary at a time: tenant/capacity -> identity/permission -> workspace/item -> path/grain/schema -> notebook/pipeline state -> serving -> monitoring/lifecycle. Compare current Microsoft Learn before changing multiple settings. If access is unavailable, stop and mark LIVE FABRIC pending.

12. You do a small version alone

Build a fresh workspace/item inventory from a blank template without looking at the guided example.

13. Validate

a workspace/item inventory naming purpose, owner, input/output and validation evidence for every item. Label every proof LIVE FABRIC, OFFLINE PLAN or SIMULATOR. A green status alone is never sufficient.

14. Teach it back

Explain fabric workspace and item model for 60–90 seconds. Name the object, the state it controls, one failure mode and one validation.

15. Protected review + changed retry

Save 04_PRACTICE_FILES/lesson_workspaces/DE15-04/attempt.md first. Only then open the protected Review. If weak, close Review and do this changed retry separately: Build a fresh workspace/item inventory from a blank template without looking at the guided example. Preserve the failed attempt append-only.

16. Evidence / mastery + exact next route

CREATE evidence/ only for genuine observed proof. Mastery requires the independent/changed proof, validation, teach-back, and no unresolved critical issue. Exact next route: DE15-05.

DE15-05 - OneLake paths, files, tables and shortcuts

1. Why this matters

OneLake exposes lakehouse Files and Tables differently. Files can hold arbitrary lake data; Tables represent managed/discoverable Delta tables. Shortcuts point to data in another OneLake or supported external location without copying the bytes. A path that looks convenient can still be wrong if it bypasses the intended table contract or security boundary. In a job, a wrong assumption here can create data loss, excess privilege, unreproducible runs, or a cloud bill with no trustworthy output.

2. What you are learning

You will explain and apply OneLake paths, files, tables and shortcuts, then produce this proof: explicit Files, Tables and shortcut paths plus a sentence explaining which bytes are copied and which are referenced.

3. Prerequisite / start state

DE15-04 mastered. Start with a blank lesson workspace; do not begin from the Review or a finished architecture plan.

4. Picture it in your head

Imagine a filesystem with two meanings: Files is general file storage; Tables is managed table space. A shortcut looks like a local doorway but points to governed data elsewhere.

5. New words before we use them

Files	Lakehouse area for general files/non-table data.
Tables	Lakehouse area for managed Delta tables.
shortcut path	Where a shortcut appears in OneLake.
target path	Data location the shortcut references.

DE15-05 - Visual + follow with me once

6. Useful video / current source route

[VIDEO FIRST: Microsoft Learn DP-700 — Ingest/transform \(Part 2\)](#)

Use: 01:49 onward — ingest/storage choices; then exact shortcut reference | Status: PRESERVED_VISUAL_ROUTE_WITH_MICROSOFT_LEARN_FALLBACK_2026-09-11

[CURRENT OFFICIAL REFERENCE: OneLake shortcuts](#)

Use: Current product behavior and exact lesson reference | Status: CURRENT_MICROSOFT_LEARN_2026-09-11

7. Follow with me once

Use the normal Retail Orders fixture or a tiny named Fabric object. Before clicking anything, write the expected object/state and whether this will be LIVE FABRIC or OFFLINE PLAN. 1. Write a Bronze Files path that preserves ingest date. 2. Write a managed Silver table name and grain. 3. Add one shortcut for governed shared reference data. 4. State which path is visible as a Delta table to SQL and which is merely file/reference storage. After each step, record the exact item/path/identity/run evidence you actually observed. If the portal or tenant is unavailable, stop the live claim, keep the design labeled OFFLINE, and use the official reference only to understand the object—not to fabricate a successful run.

```
/OneLake/<workspace>/<lakehouse>/Files/bronze/...  
/OneLake/<workspace>/<lakehouse>/Tables/silver_orders  
shortcut: /Files/reference/customers -> governed target
```

8. What you should see / be able to say

explicit Files, Tables and shortcut paths plus a sentence explaining which bytes are copied and which are referenced.

9. Why it worked

The exercise separates the Fabric object from the durable state it controls, then attaches identity, validation and failure evidence. That makes the result explainable even when the portal layout changes.

DE15-05 - Solo proof, repair and mastery

10. Common beginner mistakes

- Copying portal steps without understanding the object/state created.
- Assuming an Admin experience equals a least-privilege user experience.
- Treating a successful run badge as enough without data reconciliation.
- Hard-coding workspace IDs, paths, credentials or environment-specific values.

11. If you get stuck

Preserve the first symptom. Check one boundary at a time: tenant/capacity -> identity/permission -> workspace/item -> path/grain/schema -> notebook/pipeline state -> serving -> monitoring/lifecycle. Compare current Microsoft Learn before changing multiple settings. If access is unavailable, stop and mark LIVE FABRIC pending.

12. You do a small version alone

Design fresh OneLake Files/Tables/shortcut paths and explain how SQL/Spark would see each.

13. Validate

explicit Files, Tables and shortcut paths plus a sentence explaining which bytes are copied and which are referenced. Label every proof LIVE FABRIC, OFFLINE PLAN or SIMULATOR. A green status alone is never sufficient.

14. Teach it back

Explain onelake paths, files, tables and shortcuts for 60–90 seconds. Name the object, the state it controls, one failure mode and one validation.

15. Protected review + changed retry

Save 04_PRACTICE_FILES/lesson_workspaces/DE15-05/attempt.md first. Only then open the protected Review. If weak, close Review and do this changed retry separately: Design fresh OneLake Files/Tables/shortcut paths and explain how SQL/Spark would see each. Preserve the failed attempt append-only.

16. Evidence / mastery + exact next route

CREATE evidence/ only for genuine observed proof. Mastery requires the independent/changed proof, validation, teach-back, and no unresolved critical issue. Exact next route: DE15-06.

DE15-06 - Fabric Lakehouse and Delta tables

1. Why this matters

A Fabric Lakehouse combines open Delta storage in OneLake with Spark-based engineering and a SQL analytics endpoint for read-oriented T-SQL access. Delta tables add transaction/version metadata around Parquet files. The lakehouse is not “a warehouse with folders”; it serves mixed structured/unstructured engineering while the Warehouse is optimized for T-SQL-centric relational workloads and supports different transaction capabilities. In a job, a wrong assumption here can create data loss, excess privilege, unreproducible runs, or a cloud bill with no trustworthy output.

2. What you are learning

You will explain and apply Fabric Lakehouse and Delta tables, then produce this proof: table grain + schema contract + bad-row rules + 7 raw / 4 accepted / 3 quarantined / 3300 control for the normal fixture.

3. Prerequisite / start state

DE15-05 mastered. Start with a blank lesson workspace; do not begin from the Review or a finished architecture plan.

4. Picture it in your head

A Fabric lakehouse is durable OneLake storage plus Delta table semantics. Bronze preserves source truth, Silver enforces a usable contract, and SQL/Spark read the same table state through different engines.

5. New words before we use them

lakehouse	Analytics storage item combining lake flexibility and table semantics.
Delta table	Table backed by Parquet files plus Delta transaction metadata.
grain	What one row represents.
reconciliation	Independent comparison proving expected rows/totals/state.

DE15-06 - Visual + follow with me once

6. Useful video / current source route

[VIDEO FIRST: Microsoft Learn DP-700 — Ingest/transform \(Part 2\)](#)

Use: 01:49 onward — batch transform/storage and lakehouse context | Status: PRESERVED_VISUAL_ROUTE_WITH_MICROSOFT_LEARN_FALLBACK_2026-09-11

[CURRENT OFFICIAL REFERENCE: Fabric lakehouse overview](#)

Use: Current product behavior and exact lesson reference | Status: CURRENT_MICROSOFT_LEARN_2026-09-11

7. Follow with me once

Use the normal Retail Orders fixture or a tiny named Fabric object. Before clicking anything, write the expected object/state and whether this will be LIVE FABRIC or OFFLINE PLAN. 1. Open normal/base.csv and state raw row grain before cleaning. 2. Apply date/entity/business-state acceptance rules without changing raw source. 3. Reconcile 7 raw -> 4 accepted + 3 quarantined and accepted value 3300. 4. Write the SilverOrders table contract and one query/check you would use after write. After each step, record the exact item/path/identity/run evidence you actually observed. If the portal or tenant is unavailable, stop the live claim, keep the design labeled OFFLINE, and use the official reference only to understand the object—not to fabricate a successful run.

```
raw 7 -> accepted 4 + quarantine 3
Silver total = 3300
Delta key = order_id
Rerun same incremental input -> same final table state
```

8. What you should see / be able to say

table grain + schema contract + bad-row rules + 7 raw / 4 accepted / 3 quarantined / 3300 control for the normal fixture.

9. Why it worked

The exercise separates the Fabric object from the durable state it controls, then attaches identity, validation and failure evidence. That makes the result explainable even when the portal layout changes.

DE15-06 - Solo proof, repair and mastery

10. Common beginner mistakes

- Copying portal steps without understanding the object/state created.
- Assuming an Admin experience equals a least-privilege user experience.
- Treating a successful run badge as enough without data reconciliation.
- Hard-coding workspace IDs, paths, credentials or environment-specific values.

11. If you get stuck

Preserve the first symptom. Check one boundary at a time: tenant/capacity -> identity/permission -> workspace/item -> path/grain/schema -> notebook/pipeline state -> serving -> monitoring/lifecycle. Compare current Microsoft Learn before changing multiple settings. If access is unavailable, stop and mark LIVE FABRIC pending.

12. You do a small version alone

Define a fresh lakehouse Delta table contract and validation for a new dataset.

13. Validate

table grain + schema contract + bad-row rules + 7 raw / 4 accepted / 3 quarantined / 3300 control for the normal fixture. Label every proof LIVE FABRIC, OFFLINE PLAN or SIMULATOR. A green status alone is never sufficient.

14. Teach it back

Explain fabric lakehouse and delta tables for 60–90 seconds. Name the object, the state it controls, one failure mode and one validation.

15. Protected review + changed retry

Save 04_PRACTICE_FILES/lesson_workspaces/DE15-06/attempt.md first. Only then open the protected Review. If weak, close Review and do this changed retry separately: Define a fresh lakehouse Delta table contract and validation for a changed dataset. Preserve the failed attempt append-only.

16. Evidence / mastery + exact next route

CREATE evidence/ only for genuine observed proof. Mastery requires the independent/changed proof, validation, teach-back, and no unresolved critical issue. Exact next route: DE15-07.

DE15-07 - Fabric notebooks

1. Why this matters

Fabric notebooks are browser-based code items used for Spark/Python/SQL workflows. Attach the intended lakehouse, make inputs/outputs explicit, separate parameters/config from business logic, and make reruns safe. A notebook that only works because cell 7 was manually run yesterday is not a production data pipeline. In a job, a wrong assumption here can create data loss, excess privilege, unreproducible runs, or a cloud bill with no trustworthy output.

2. What you are learning

You will explain and apply Fabric notebooks, then produce this proof: rerunnable notebook flow with parameters/config separated from transformation logic and a clean-start validation plan.

3. Prerequisite / start state

DE15-06 mastered. Start with a blank lesson workspace; do not begin from the Review or a finished architecture plan.

4. Picture it in your head

A notebook is an executable lab notebook, not durable orchestration by itself. Cells are code steps; parameters/config separate environment from logic; rerunnable design makes a second run safe.

5. New words before we use them

notebook	Interactive executable document containing code/markdown cells.
parameter	Input value supplied without editing business logic.
session	Active compute context for notebook execution.
rerunnable	Safe to run again from a defined starting state.

DE15-07 - Visual + follow with me once

6. Useful video / current source route

[VIDEO FIRST: Microsoft Learn DP-700 — Ingest/transform \(Part 2\)](#)

Use: 01:49 onward — transformation choices; then notebook UI/reference | Status: PRESERVED_VISUAL_ROUTE_WITH_MICROSOFT_LEARN_FALLBACK_2026-09-11

[CURRENT OFFICIAL REFERENCE: How to use Fabric notebooks](#)

Use: Current product behavior and exact lesson reference | Status: CURRENT_MICROSOFT_LEARN_2026-09-11

7. Follow with me once

Use the normal Retail Orders fixture or a tiny named Fabric object. Before clicking anything, write the expected object/state and whether this will be LIVE FABRIC or OFFLINE PLAN. 1. Start with a blank notebook-flow list. 2. Add parameters/config before any data read. 3. Add read -> validate -> transform -> write -> reconcile in that order and mark which cells may have side effects. 4. Describe a clean-session rerun test and what evidence would fail it. After each step, record the exact item/path/identity/run evidence you actually observed. If the portal or tenant is unavailable, stop the live claim, keep the design labeled OFFLINE, and use the official reference only to understand the object—not to fabricate a successful run.

```
params/config -> read -> validate -> transform -> write -> reconcile
# second run must start from declared state, not hidden notebook memory
```

8. What you should see / be able to say

rerunnable notebook flow with parameters/config separated from transformation logic and a clean-start validation plan.

9. Why it worked

The exercise separates the Fabric object from the durable state it controls, then attaches identity, validation and failure evidence. That makes the result explainable even when the portal layout changes.

DE15-07 - Solo proof, repair and mastery

10. Common beginner mistakes

- Copying portal steps without understanding the object/state created.
- Assuming an Admin experience equals a least-privilege user experience.
- Treating a successful run badge as enough without data reconciliation.
- Hard-coding workspace IDs, paths, credentials or environment-specific values.

11. If you get stuck

Preserve the first symptom. Check one boundary at a time: tenant/capacity -> identity/permission -> workspace/item -> path/grain/schema -> notebook/pipeline state -> serving -> monitoring/lifecycle. Compare current Microsoft Learn before changing multiple settings. If access is unavailable, stop and mark LIVE FABRIC pending.

12. You do a small version alone

Design a rerunnable notebook flow from a blank file: parameters → read → validate → transform → write → reconcile.

13. Validate

rerunnable notebook flow with parameters/config separated from transformation logic and a clean-start validation plan. Label every proof LIVE FABRIC, OFFLINE PLAN or SIMULATOR. A green status alone is never sufficient.

14. Teach it back

Explain fabric notebooks for 60–90 seconds. Name the object, the state it controls, one failure mode and one validation.

15. Protected review + changed retry

Save 04_PRACTICE_FILES/lesson_workspaces/DE15-07/attempt.md first. Only then open the protected Review. If weak, close Review and do this changed retry separately: Design a rerunnable notebook flow from a blank file: parameters → read → validate → transform → write → reconcile. Preserve the failed attempt append-only.

16. Evidence / mastery + exact next route

CREATE evidence/ only for genuine observed proof. Mastery requires the independent/changed proof, validation, teach-back, and no unresolved critical issue. Exact next route: DE15-08.

DE15-08 - Spark in Fabric

1. Why this matters

Fabric Runtime is managed separately from upstream open-source Spark. At this rebuild checkpoint, Fabric Runtime 1.3 is the latest GA/default runtime (Spark 3.5.5, Python 3.11, Java 11, Delta 3.2), while Runtime 2.0 is GA (opt-in) (Spark 4.1, Python 3.13, Java 21, Delta 4.2). Our local M12/M13 course baselines differ intentionally. Test code against the runtime where it will execute; a version-family label alone is not a compatibility guarantee.

2. What you are learning

You will explain and apply Spark in Fabric, then produce this proof: compatibility checklist that explicitly records local Spark 4.2/Delta 4.4 versus Fabric Runtime 2.0 Spark 4.1/Delta 4.2.

3. Prerequisite / start state

DE15-07 mastered. Start with a blank lesson workspace; do not begin from the Review or a finished architecture plan.

4. Picture it in your head

Fabric Spark is managed Spark running inside a selected Fabric Runtime. The code idea may resemble local Spark, but runtime versions, libraries, capacity and platform optimizations are a separate environment boundary.

5. New words before we use them

Fabric Runtime	Managed Spark software stack selected for Fabric workloads.
GA	Generally available production-supported release stage.
preview	Pre-GA feature/runtime stage that can change.
environment item	Fabric configuration item for runtime/libraries/settings.

DE15-08 - Visual + follow with me once

6. Useful video / current source route

[VIDEO FIRST: Microsoft Learn DP-700 — Ingest/transform \(Part 2\)](#)

Use: 01:49 onward — Spark transform context; current Fabric Runtime doc controls versions | Status: PRESERVED_VISUAL_ROUTE_WITH_MICROSOFT_LEARN_FALLBACK_2026-09-11

[CU](#)

Use:

| Status:

CUR

[CU](#)

Use:

| Status:

CUR

7. Follow with me once

Use the normal Retail Orders fixture or a tiny named Fabric object. Before clicking anything, write the expected object/state and whether this will be LIVE FABRIC or OFFLINE PLAN. 1. Write LOCAL and FABRIC columns. 2. Record local Spark 4.2 / Delta 4.4 and Fabric Runtime 2.0 Spark 4.1 / Delta 4.2. 3. List APIs/packages/functions your project actually relies on. 4. For each dependency write a compatibility test or fallback before promotion. After each step, record the exact item/path/identity/run evidence you actually observed. If the portal or tenant is unavailable, stop the live claim, keep the design labeled OFFLINE, and use the official reference only to understand the object—not to fabricate a successful run.

Production/default today: Runtime 1.3 GA

Spark 3.5.5 | Python 3.11 | Java 11 | Delta 3.2

21 | Delta 4.2

8. What you should see / be able to say

compatibility checklist that explicitly records local Spark 4.2/Delta 4.4 versus Fabric Runtime 2.0 Spark 4.1/Delta 4.2.

9. Why it worked

The exercise separates the Fabric object from the durable state it controls, then attaches identity, validation and failure evidence. That makes the result explainable even when the portal layout changes.

DE15-08 - Solo proof, repair and mastery

10. Common beginner mistakes

- Copying portal steps without understanding the object/state created.
- Assuming an Admin experience equals a least-privilege user experience.
- Treating a successful run badge as enough without data reconciliation.
- Hard-coding workspace IDs, paths, credentials or environment-specific values.

11. If you get stuck

Preserve the first symptom. Check one boundary at a time: tenant/capacity -> identity/permission -> workspace/item -> path/grain/schema -> notebook/pipeline state -> serving -> monitoring/lifecycle. Compare current Microsoft Learn before changing multiple settings. If access is unavailable, stop and mark LIVE FABRIC pending.

12. You do a small version alone

Write a runtime-compatibility checklist for moving local Spark/Delta code into Fabric Runtime 2.0.

13. Validate

compatibility checklist that explicitly records local Spark 4.2/Delta 4.4 versus Fabric Runtime 2.0 Spark 4.1/Delta 4.2. Label every proof LIVE FABRIC, OFFLINE PLAN or SIMULATOR. A green status alone is never sufficient.

14. Teach it back

Explain spark in fabric for 60–90 seconds. Name the object, the state it controls, one failure mode and one validation.

15. Protected review + changed retry

Save 04_PRACTICE_FILES/lesson_workspaces/DE15-08/attempt.md first. Only then open the protected Review. If weak, close Review and do this changed retry separately: Write a runtime-compatibility checklist for moving local Spark/Delta code into Fabric Runtime 2.0. Preserve the failed attempt append-only.

16. Evidence / mastery + exact next route

CREATE evidence/ only for genuine observed proof. Mastery requires the independent/changed proof, validation, teach-back, and no unresolved critical issue. Exact next route: DE15-09.

DE15-09 - Fabric Data Pipelines

1. Why this matters

Fabric Data Pipelines orchestrate activities such as copy, notebook and other jobs. A pipeline is control flow: dependency, parameters, retries, failure routing and schedules. Data movement itself can use Copy activity/Copy job and other mechanisms. A pipeline is trustworthy only when a failed upstream activity prevents a misleading downstream publish and reruns are idempotent. In a job, a wrong assumption here can create data loss, excess privilege, unreproducible runs, or a cloud bill with no trustworthy output.

2. What you are learning

You will explain and apply Fabric Data Pipelines, then produce this proof: dependency graph, parameter, safe-retry decision, blocking failure route and checkpoint/publish rule.

3. Prerequisite / start state

DE15-08 mastered. Start with a blank lesson workspace; do not begin from the Review or a finished architecture plan.

4. Picture it in your head

A pipeline is a workflow conductor coordinating activities and dependencies. A Copy job is a simpler data-movement item. Use orchestration when multiple dependent steps/failure paths must be controlled.

5. New words before we use them

pipeline	Orchestrated group of activities and dependencies.
activity	A unit of pipeline work.
Copy job	Simplified Fabric item for data movement, including incremental/CDC patterns.
dependency	Rule controlling which activity may run next.

DE15-09 - Visual + follow with me once

6. Useful video / current source route

[VIDEO FIRST: Microsoft Learn DP-700 — Implement/manage \(Part 1\)](#)

Use: 17:00 onward — orchestration processes | Status: PRESERVED_VISUAL_ROUTE_WITH_MICROSOFT_LEARN_FALLBACK_2026-09-11

[CURRENT OFFICIAL REFERENCE: Fabric Data Factory — Copy job](#)

Use: Current product behavior and exact lesson reference | Status: CURRENT_MICROSOFT_LEARN_2026-09-11

[CURRENT OFFICIAL REFERENCE: Fabric Data Factory pipeline overview](#)

Use: Current orchestration model; Copy job remains a distinct simplified data-movement item | Status: CURRENT_MICROSOFT_LEARN_2026-09-11

7. Follow with me once

Use the normal Retail Orders fixture or a tiny named Fabric object. Before clicking anything, write the expected object/state and whether this will be LIVE FABRIC or OFFLINE PLAN. 1. Draw Copy Bronze -> Notebook Silver -> After each step, record the exact item/path/identity/run evidence you actually observed. If the portal or tenant is unavailable, stop the live claim, keep the design labeled OFFLINE, and use the official reference only to understand the object—not to fabricate a successful run.

```
Pipeline: ingest -> validate -> notebook -> publish
          failure -> preserve state -> retry/repair
Copy job: focused data movement (full/incremental/CDC)
```

8. What you should see / be able to say

dependency graph, parameter, safe-retry decision, blocking failure route and checkpoint/publish rule.

9. Why it worked

The exercise separates the Fabric object from the durable state it controls, then attaches identity, validation and failure evidence. That makes the result explainable even when the portal layout changes.

DE15-09 - Solo proof, repair and mastery

10. Common beginner mistakes

- Copying portal steps without understanding the object/state created.
- Assuming an Admin experience equals a least-privilege user experience.
- Treating a successful run badge as enough without data reconciliation.
- Hard-coding workspace IDs, paths, credentials or environment-specific values.

11. If you get stuck

Preserve the first symptom. Check one boundary at a time: tenant/capacity -> identity/permission -> workspace/item -> path/grain/schema -> notebook/pipeline state -> serving -> monitoring/lifecycle. Compare current Microsoft Learn before changing multiple settings. If access is unavailable, stop and mark LIVE FABRIC pending.

12. You do a small version alone

Design an independent pipeline with at least four activities, dependency, parameter, retry and failure branch.

13. Validate

dependency graph, parameter, safe-retry decision, blocking failure route and checkpoint/publish rule. Label every proof LIVE FABRIC, OFFLINE PLAN or SIMULATOR. A green status alone is never sufficient.

14. Teach it back

Explain fabric data pipelines for 60–90 seconds. Name the object, the state it controls, one failure mode and one validation.

15. Protected review + changed retry

Save 04_PRACTICE_FILES/lesson_workspaces/DE15-09/attempt.md first. Only then open the protected Review. If weak, close Review and do this changed retry separately: Design an independent pipeline with at least four activities, dependency, parameter, retry and failure branch. Preserve the failed attempt append-only.

16. Evidence / mastery + exact next route

CREATE evidence/ only for genuine observed proof. Mastery requires the independent/changed proof, validation, teach-back, and no unresolved critical issue. Exact next route: DE15-10.

DE15-10 - SQL analytics endpoint and warehouse choice

1. Why this matters

Every Fabric lakehouse automatically gets a SQL analytics endpoint: a read-only T-SQL surface over Delta tables. It is excellent for validation, ad-hoc analysis and downstream SQL consumption, but it is not the place to mutate lakehouse Delta data. Choose Warehouse when your workload is strongly relational/T-SQL-centric and needs warehouse capabilities; choose Lakehouse when Spark/open-file engineering is central. In a job, a wrong assumption here can create data loss, excess privilege, unreproducible runs, or a cloud bill with no trustworthy output.

2. What you are learning

You will explain and apply SQL analytics endpoint and warehouse choice, then produce this proof: two read-only validation queries plus a justified lakehouse-vs-Warehouse choice for changed workloads.

3. Prerequisite / start state

DE15-09 mastered. Start with a blank lesson workspace; do not begin from the Review or a finished architecture plan.

4. Picture it in your head

The lakehouse SQL analytics endpoint is a read-only T-SQL window over Delta tables. A Warehouse is a different item for fuller relational/warehouse write patterns. Choose by workload, not by familiar SQL syntax alone.

5. New words before we use them

SQL analytics endpoint	Read-only T-SQL query surface over lakehouse Delta tables.
warehouse	Fabric relational analytics item with broader T-SQL warehouse behavior.
TDS	Protocol used by SQL clients to connect.
serving layer	Interface optimized for downstream query/analytics consumption.

DE15-10 - Visual + follow with me once

6. Useful video / current source route

[VIDEO FIRST: Microsoft Learn DP-700 — Ingest/transform \(Part 2\)](#)

Use: 01:49 onward — storage/serving choices; then exact SQL endpoint reference | Status: PRESERVED_VISUAL_ROUTE_WITH_MICROSOFT_LEARN_FALLBACK_2026-09-11

[CURRENT OFFICIAL REFERENCE: Lakehouse SQL analytics endpoint](#)

Use: Current product behavior and exact lesson reference | Status: CURRENT_MICROSOFT_LEARN_2026-09-11

7. Follow with me once

Use the normal Retail Orders fixture or a tiny named Fabric object. Before clicking anything, write the expected object/state and whether this will be LIVE FABRIC or OFFLINE PLAN. 1. Write a SELECT count and SUM validation query for Silver/Gold. 2. Mark them read-only and identify one UPDATE/MERGE operation that belongs on a write-capable path. 3. Classify three workloads as lakehouse endpoint or Warehouse. 4. Defend each classification using workload requirements rather than UI preference. After each step, record the exact item/path/identity/run evidence you actually observed. If the portal or tenant is unavailable, stop the live claim, keep the design labeled OFFLINE, and use the official reference only to understand the object—not to fabricate a successful run.

Lakehouse Delta table --READ--> SQL analytics endpoint --SELECT--> consumer
Writes/updates to lakehouse table -> Spark/lakehouse path, not endpoint DML

8. What you should see / be able to say

two read-only validation queries plus a justified lakehouse-vs-Warehouse choice for changed workloads.

9. Why it worked

The exercise separates the Fabric object from the durable state it controls, then attaches identity, validation and failure evidence. That makes the result explainable even when the portal layout changes.

DE15-10 - Solo proof, repair and mastery

10. Common beginner mistakes

- Copying portal steps without understanding the object/state created.
- Assuming an Admin experience equals a least-privilege user experience.
- Treating a successful run badge as enough without data reconciliation.
- Hard-coding workspace IDs, paths, credentials or environment-specific values.

11. If you get stuck

Preserve the first symptom. Check one boundary at a time: tenant/capacity -> identity/permission -> workspace/item -> path/grain/schema -> notebook/pipeline state -> serving -> monitoring/lifecycle. Compare current Microsoft Learn before changing multiple settings. If access is unavailable, stop and mark LIVE FABRIC pending.

12. You do a small version alone

Choose lakehouse SQL endpoint vs Warehouse for three changed workloads and defend each choice.

13. Validate

two read-only validation queries plus a justified lakehouse-vs-Warehouse choice for changed workloads. Label every proof LIVE FABRIC, OFFLINE PLAN or SIMULATOR. A green status alone is never sufficient.

14. Teach it back

Explain sql analytics endpoint and warehouse choice for 60–90 seconds. Name the object, the state it controls, one failure mode and one validation.

15. Protected review + changed retry

Save 04_PRACTICE_FILES/lesson_workspaces/DE15-10/attempt.md first. Only then open the protected Review. If weak, close Review and do this changed retry separately: Choose lakehouse SQL endpoint vs Warehouse for three changed workloads and defend each choice. Preserve the failed attempt append-only.

16. Evidence / mastery + exact next route

CREATE evidence/ only for genuine observed proof. Mastery requires the independent/changed proof, validation, teach-back, and no unresolved critical issue. Exact next route: DE15-11.

DE15-11 - Medallion architecture in Fabric

1. Why this matters

Medallion architecture is a contract pattern, not three folder names. Bronze preserves raw/landing truth, Silver applies validated/conformed business-ready structure, and Gold serves curated aggregates/products. Each layer needs a grain, acceptance rules, lineage and reconciliation. Do not silently “clean” Bronze until you can no longer reproduce what arrived. In a job, a wrong assumption here can create data loss, excess privilege, unreproducible runs, or a cloud bill with no trustworthy output.

2. What you are learning

You will explain and apply Medallion architecture in Fabric, then produce this proof: Bronze/Silver/Gold contracts with layer grain, acceptance/rejection behavior, lineage and reconciliation.

3. Prerequisite / start state

DE15-10 mastered. Start with a blank lesson workspace; do not begin from the Review or a finished architecture plan.

4. Picture it in your head

Medallion is a contract chain: Bronze keeps what arrived, Silver makes it trustworthy and typed, Gold shapes it for a consumer. Each boundary has a grain, acceptance rule, and reconciliation proof.

5. New words before we use them

Bronze	Raw/landing layer preserving source evidence.
Silver	Validated, conformed, typed engineering layer.
Gold	Consumer-oriented curated layer.
contract	Declared grain/schema/quality behavior at a boundary.

DE15-11 - Visual + follow with me once

6. Useful video / current source route

[VIDEO FIRST: Microsoft Learn DP-700 — Ingest/transform \(Part 2\)](#)

Use: 01:49 onward — batch architecture context; then medallion tutorial | Status: PRESERVED_VISUAL_ROUTE_WITH_MICROSOFT_LEARN_FALLBACK_2026-09-11

[CURRENT OFFICIAL REFERENCE: Lakehouse end-to-end medallion scenario](#)

Use: Current product behavior and exact lesson reference | Status: CURRENT_MICROSOFT_LEARN_2026-09-11

7. Follow with me once

Use the normal Retail Orders fixture or a tiny named Fabric object. Before clicking anything, write the expected object/state and whether this will be LIVE FABRIC or OFFLINE PLAN. 1. Keep all raw normal rows in Bronze including the three invalid business rows. 2. Apply acceptance rules to produce 4 Silver rows and a quarantine set. 3. Define one Gold aggregate at a stated grain. 4. Reconcile Bronze -> Silver+quarantine -> Gold and write the lineage sentence. After each step, record the exact item/path/identity/run evidence you actually observed. If the portal or tenant is unavailable, stop the live claim, keep the design labeled OFFLINE, and use the official reference only to understand the object—not to fabricate a successful run.

Bronze grain: source record as received

Silver grain: one valid business event/order

Gold grain: declared consumer aggregate

Every arrow: rows + business total + rejected rows

8. What you should see / be able to say

Bronze/Silver/Gold contracts with layer grain, acceptance/rejection behavior, lineage and reconciliation.

9. Why it worked

The exercise separates the Fabric object from the durable state it controls, then attaches identity, validation and failure evidence. That makes the result explainable even when the portal layout changes.

DE15-11 - Solo proof, repair and mastery

10. Common beginner mistakes

- Copying portal steps without understanding the object/state created.
- Assuming an Admin experience equals a least-privilege user experience.
- Treating a successful run badge as enough without data reconciliation.
- Hard-coding workspace IDs, paths, credentials or environment-specific values.

11. If you get stuck

Preserve the first symptom. Check one boundary at a time: tenant/capacity -> identity/permission -> workspace/item -> path/grain/schema -> notebook/pipeline state -> serving -> monitoring/lifecycle. Compare current Microsoft Learn before changing multiple settings. If access is unavailable, stop and mark LIVE FABRIC pending.

12. You do a small version alone

Define Bronze/Silver/Gold contracts for a changed domain with explicit grains and rejected-row handling.

13. Validate

Bronze/Silver/Gold contracts with layer grain, acceptance/rejection behavior, lineage and reconciliation. Label every proof LIVE FABRIC, OFFLINE PLAN or SIMULATOR. A green status alone is never sufficient.

14. Teach it back

Explain medallion architecture in fabric for 60–90 seconds. Name the object, the state it controls, one failure mode and one validation.

15. Protected review + changed retry

Save 04_PRACTICE_FILES/lesson_workspaces/DE15-11/attempt.md first. Only then open the protected Review. If weak, close Review and do this changed retry separately: Define Bronze/Silver/Gold contracts for a changed domain with explicit grains and rejected-row handling. Preserve the failed attempt append-only.

16. Evidence / mastery + exact next route

CREATE evidence/ only for genuine observed proof. Mastery requires the independent/changed proof, validation, teach-back, and no unresolved critical issue. Exact next route: DE15-12.

DE15-12 - Incremental loads and MERGE in cloud

1. Why this matters

Incremental loading moves only new/changed data after a known successful state. Watermarks, CDC and MERGE/upsert are mechanisms, not the goal. The safe rule is atomicity between published output and progress state: a failed load must not advance the checkpoint. Fabric Copy job can manage incremental state; notebook/Delta MERGE can implement explicit upsert logic when you need more control. In a job, a wrong assumption here can create data loss, excess privilege, unreproducible runs, or a cloud bill with no trustworthy output.

2. What you are learning

You will explain and apply Incremental loads and MERGE in cloud, then produce this proof: base controls, update/insert controls, 5 rows / 4250 after merge, exact-rerun no-op and failed-publish checkpoint rule.

3. Prerequisite / start state

DE15-11 mastered. Start with a blank lesson workspace; do not begin from the Review or a finished architecture plan.

4. Picture it in your head

Incremental loading is a bookmark plus idempotent state transition. Read only new/changed source state, validate it, merge by stable key, then advance the bookmark only after successful publish.

5. New words before we use them

watermark	Progress value identifying already processed source state.
CDC	Change data capture representing inserts/updates/deletes.
MERGE	Keyed insert/update state transition.
idempotent	Same input repeated produces no unintended new effect.

DE15-12 - Visual + follow with me once

6. Useful video / current source route

[VIDEO FIRST: Microsoft Learn DP-700 — Ingest/transform \(Part 2\)](#)

Use: 00:47 onward — loading patterns; then current incremental-copy reference | Status: PRESERVED_VISUAL_ROUTE_WITH_MICROSOFT_LEARN_FALLBACK_2026-09-11

[CURRENT OFFICIAL REFERENCE: Fabric Copy job incremental copy](#)

Use: Current product behavior and exact lesson reference | Status: CURRENT_MICROSOFT_LEARN_2026-09-11

7. Follow with me once

Use the normal Retail Orders fixture or a tiny named Fabric object. Before clicking anything, write the expected object/state and whether this will be LIVE FABRIC or OFFLINE PLAN. 1. Run the bundled normal offline simulator and save observed_controls.json. 2. Explain O2 as update and O8 as insert; verify 5 rows / 4250 after merge. 3. Run it again and prove the same result does not create another side effect. 4. Describe an injected publish failure and write the prior checkpoint value that must remain durable. After each step, record the exact item/path/identity/run evidence you actually observed. If the portal or tenant is unavailable, stop the live claim, keep the design labeled OFFLINE, and use the official reference only to understand the object—not to fabricate a successful run.

```
last_successful_watermark -> read changes -> validate -> MERGE -> reconcile ->
publish -> advance watermark
FAIL before publish -> keep prior successful watermark
```

8. What you should see / be able to say

base controls, update/insert controls, 5 rows / 4250 after merge, exact-rerun no-op and failed-publish checkpoint rule.

9. Why it worked

The exercise separates the Fabric object from the durable state it controls, then attaches identity, validation and failure evidence. That makes the result explainable even when the portal layout changes.

DE15-12 - Solo proof, repair and mastery

10. Common beginner mistakes

- Copying portal steps without understanding the object/state created.
- Assuming an Admin experience equals a least-privilege user experience.
- Treating a successful run badge as enough without data reconciliation.
- Hard-coding workspace IDs, paths, credentials or environment-specific values.

11. If you get stuck

Preserve the first symptom. Check one boundary at a time: tenant/capacity -> identity/permission -> workspace/item -> path/grain/schema -> notebook/pipeline state -> serving -> monitoring/lifecycle. Compare current Microsoft Learn before changing multiple settings. If access is unavailable, stop and mark LIVE FABRIC pending.

12. You do a small version alone

Design a new incremental load where a failed publish must not advance progress. State the atomic boundary.

13. Validate

base controls, update/insert controls, 5 rows / 4250 after merge, exact-rerun no-op and failed-publish checkpoint rule. Label every proof LIVE FABRIC, OFFLINE PLAN or SIMULATOR. A green status alone is never sufficient.

14. Teach it back

Explain incremental loads and merge in cloud for 60–90 seconds. Name the object, the state it controls, one failure mode and one validation.

15. Protected review + changed retry

Save 04_PRACTICE_FILES/lesson_workspaces/DE15-12/attempt.md first. Only then open the protected Review. If weak, close Review and do this changed retry separately: Design a changed incremental load where a failed publish must not advance progress. State the atomic boundary. Preserve the failed attempt append-only.

16. Evidence / mastery + exact next route

CREATE evidence/ only for genuine observed proof. Mastery requires the independent/changed proof, validation, teach-back, and no unresolved critical issue. Exact next route: DE15-13.

DE15-13 - Permissions, connections and secrets

1. Why this matters

Connections centralize access to external systems and credentials. Never paste secrets into notebooks, Git, screenshots or logs. Separate workspace/item permissions from data access and from external-source credentials. OneLake security can scope data access more finely than a broad workspace role. Verify with a least-privilege test identity rather than assuming permissions from your own admin account. In a job, a wrong assumption here can create data loss, excess privilege, unreproducible runs, or a cloud bill with no trustworthy output.

2. What you are learning

You will explain and apply Permissions, connections and secrets, then produce this proof: identity -> permission -> resource matrix, no literal secret, and evidence for one allowed plus one denied access path.

3. Prerequisite / start state

DE15-12 mastered. Start with a blank lesson workspace; do not begin from the Review or a finished architecture plan.

4. Picture it in your head

Separate three security layers in your head: Fabric identity, permission to the workspace/item/data, and credential/connection to an external system. A secret is never the same thing as a role.

5. New words before we use them

connection	Managed configuration for reaching an external system.
secret	Sensitive credential value kept out of source code/logs.
control plane	Permissions governing items/configuration/actions.
data plane	Permissions governing data read/write access.

DE15-13 - Visual + follow with me once

6. Useful video / current source route

[VIDEO FIRST: Microsoft Learn DP-700 — Implement/manage \(Part 1\)](#)

Use: Use the Implement/manage security/connection portion; current OneLake security docs control permissions | Status: PRESERVED_VISUAL_ROUTE_WITH_MICROSOFT_LEARN_FALLBACK_2026-09-11

[CURRENT OFFICIAL REFERENCE: OneLake data security overview](#)

Use: Current product behavior and exact lesson reference | Status: CURRENT_MICROSOFT_LEARN_2026-09-11

7. Follow with me once

Use the normal Retail Orders fixture or a tiny named Fabric object. Before clicking anything, write the expected object/state and whether this will be LIVE FABRIC or OFFLINE PLAN. 1. Create identities for engineer, analyst and pipeline. 2. Separate workspace/item permissions, OneLake data permissions and external connection credential. 3. Replace any literal credential with a named connection/secret reference. 4. Write one allowed and one denied least-privilege test using a non-admin identity. After each step, record the exact item/path/identity/run evidence you actually observed. If the portal or tenant is unavailable, stop the live claim, keep the design labeled OFFLINE, and use the official reference only to understand the object—not to fabricate a successful run.

Entra identity -> Fabric workspace/item permission -> OneLake data role
External source -> named connection -> secret store/credential
Never: token = "literal-value-in-notebook"

8. What you should see / be able to say

identity -> permission -> resource matrix, no literal secret, and evidence for one allowed plus one denied access path.

9. Why it worked

The exercise separates the Fabric object from the durable state it controls, then attaches identity, validation and failure evidence. That makes the result explainable even when the portal layout changes.

DE15-13 - Solo proof, repair and mastery

10. Common beginner mistakes

- Copying portal steps without understanding the object/state created.
- Assuming an Admin experience equals a least-privilege user experience.
- Treating a successful run badge as enough without data reconciliation.
- Hard-coding workspace IDs, paths, credentials or environment-specific values.

11. If you get stuck

Preserve the first symptom. Check one boundary at a time: tenant/capacity -> identity/permission -> workspace/item -> path/grain/schema -> notebook/pipeline state -> serving -> monitoring/lifecycle. Compare current Microsoft Learn before changing multiple settings. If access is unavailable, stop and mark LIVE FABRIC pending.

12. You do a small version alone

Find and repair three security mistakes in the changed plan: broad role, hard-coded secret and unclear external connection identity.

13. Validate

identity -> permission -> resource matrix, no literal secret, and evidence for one allowed plus one denied access path. Label every proof LIVE FABRIC, OFFLINE PLAN or SIMULATOR. A green status alone is never sufficient.

14. Teach it back

Explain permissions, connections and secrets for 60–90 seconds. Name the object, the state it controls, one failure mode and one validation.

15. Protected review + changed retry

Save 04_PRACTICE_FILES/lesson_workspaces/DE15-13/attempt.md first. Only then open the protected Review. If weak, close Review and do this changed retry separately: Find and repair three security mistakes in the changed plan: broad role, hard-coded secret and unclear external connection identity. Preserve the failed attempt append-only.

16. Evidence / mastery + exact next route

CREATE evidence/ only for genuine observed proof. Mastery requires the independent/changed proof, validation, teach-back, and no unresolved critical issue. Exact next route: DE15-14.

DE15-14 - Monitoring, reliability and cost awareness

1. Why this matters

Cloud reliability is observable behavior: run status, duration, freshness, row/value reconciliation, failure details and capacity consumption. Fabric Monitoring hub centralizes activity health; Capacity Metrics helps capacity administrators understand consumption. Cost awareness means selecting efficient load patterns and compute—not chasing the cheapest number while creating unreliable reruns. In a job, a wrong assumption here can create data loss, excess privilege, unreproducible runs, or a cloud bill with no trustworthy output.

2. What you are learning

You will explain and apply Monitoring, reliability and cost awareness, then produce this proof: monitoring record covering run status, freshness, quality/reconciliation, duration, failure detail and capacity/cost signal.

3. Prerequisite / start state

DE15-13 mastered. Start with a blank lesson workspace; do not begin from the Review or a finished architecture plan.

4. Picture it in your head

Monitoring is a flight recorder: status alone says the plane landed, not whether the cargo is correct. Track freshness, row/value controls, duration, failures and capacity/cost together.

5. New words before we use them

freshness	How current the available data is.
capacity unit	Fabric consumption resource accounting unit.
run ID	Identifier for one execution instance.
SLA	Expected service level such as completion/freshness target.

DE15-14 - Visual + follow with me once

6. Useful video / current source route

[VIDEO FIRST: Microsoft Learn DP-700 — Monitor/optimize \(Part 3\)](#)

Use: 00:49 monitoring • 02:03 alerts • 10:01 optimization | Status: PRESERVED_VISUAL_ROUTE_WITH_MICROSOFT_LEARN_FALLBACK_2026-09-11

[CURRENT OFFICIAL REFERENCE: Fabric monitoring hub](#)

Use: Current product behavior and exact lesson reference | Status: CURRENT_MICROSOFT_LEARN_2026-09-11

[CURRENT OFFICIAL REFERENCE: Fabric pipeline pricing](#)

Use: CU-based pipeline consumption and Capacity Metrics cost estimation | Status: CURRENT_MICROSOFT_LEARN_2026-09-11

7. Follow with me once

Use the normal Retail Orders fixture or a tiny named Fabric object. Before clicking anything, write the expected object/state and whether this will be LIVE FABRIC or OFFLINE PLAN. 1. Create three run records: quality anomaly, auth failure, duration/capacity spike. 2. For each record write the first evidence you inspect. 3. Choose owner + recovery action + downstream validation. 4. Add freshness and cost/capacity signal so success/failure is not the only telemetry. After each step, record the exact item/path/identity/run evidence you actually observed. If the portal or tenant is unavailable, stop the live claim, keep the design labeled OFFLINE, and use the official reference only to understand the object—not to fabricate a successful run.

```
run_id | status | freshness | rows/value | duration | failure | CU signal
A green status with wrong rows is still a bad run.
```

8. What you should see / be able to say

monitoring record covering run status, freshness, quality/reconciliation, duration, failure detail and capacity/cost signal.

9. Why it worked

The exercise separates the Fabric object from the durable state it controls, then attaches identity, validation and failure evidence. That makes the result explainable even when the portal layout changes.

DE15-14 - Solo proof, repair and mastery

10. Common beginner mistakes

- Copying portal steps without understanding the object/state created.
- Assuming an Admin experience equals a least-privilege user experience.
- Treating a successful run badge as enough without data reconciliation.
- Hard-coding workspace IDs, paths, credentials or environment-specific values.

11. If you get stuck

Preserve the first symptom. Check one boundary at a time: tenant/capacity -> identity/permission -> workspace/item -> path/grain/schema -> notebook/pipeline state -> serving -> monitoring/lifecycle. Compare current Microsoft Learn before changing multiple settings. If access is unavailable, stop and mark LIVE FABRIC pending.

12. You do a small version alone

Create a monitoring checklist covering freshness, quality, failure, duration and capacity/cost signals.

13. Validate

monitoring record covering run status, freshness, quality/reconciliation, duration, failure detail and capacity/cost signal. Label every proof LIVE FABRIC, OFFLINE PLAN or SIMULATOR. A green status alone is never sufficient.

14. Teach it back

Explain monitoring, reliability and cost awareness for 60–90 seconds. Name the object, the state it controls, one failure mode and one validation.

15. Protected review + changed retry

Save 04_PRACTICE_FILES/lesson_workspaces/DE15-14/attempt.md first. Only then open the protected Review. If weak, close Review and do this changed retry separately: Create a monitoring checklist covering freshness, quality, failure, duration and capacity/cost signals. Preserve the failed attempt append-only.

16. Evidence / mastery + exact next route

CREATE evidence/ only for genuine observed proof. Mastery requires the independent/changed proof, validation, teach-back, and no unresolved critical issue. Exact next route: DE15-15.

DE15-15 - Git, deployment and environment separation

1. Why this matters

Git integration versions supported Fabric item definitions; deployment pipelines promote content through environments. Dev/Test/Prod should differ through configuration, not manual edits inside copied notebooks. Source control is not deployment by itself. A safe release has review, environment-specific variables/connections, validation after deploy, and a rollback/recovery plan. In a job, a wrong assumption here can create data loss, excess privilege, unreproducible runs, or a cloud bill with no trustworthy output.

2. What you are learning

You will explain and apply Git, deployment and environment separation, then produce this proof: dev/test/prod mapping, Git branch/review path, environment configuration separation, validation-after-deploy and rollback plan.

3. Prerequisite / start state

DE15-14 mastered. Start with a blank lesson workspace; do not begin from the Review or a finished architecture plan.

4. Picture it in your head

Git versions supported item definitions; deployment moves reviewed content between environments; configuration supplies environment-specific IDs/connections. Source control and deployment are related but not identical.

5. New words before we use them

Git integration	Workspace sync with supported Git provider.
deployment pipeline	Promotion path across Fabric environments/stages.
configuration separation	Environment values kept separate from shared logic.
rollback	Controlled return/recovery after bad deployment.

DE15-15 - Visual + follow with me once

6. Useful video / current source route

[VIDEO FIRST: Microsoft Learn DP-700 — Implement/manage \(Part 1\)](#)

Use: 06:22 onward — lifecycle management in Fabric | Status: PRESERVED_VISUAL_ROUTE_WITH_MICROSOFT_LEARN_FALLBACK_2026-09-11

[CURRENT OFFICIAL REFERENCE: Fabric CI/CD overview](#)

Use: Current product behavior and exact lesson reference | Status: CURRENT_MICROSOFT_LEARN_2026-09-11

[CURRENT OFFICIAL REFERENCE: Get started with Fabric Git integration](#)

Use: Connect workspace to Git and sync lifecycle | Status: CURRENT_MICROSOFT_LEARN_2026-09-11

[CURRENT OFFICIAL REFERENCE: Fabric lifecycle best practices](#)

Use: Dev/Test/Prod lifecycle guidance | Status: CURRENT_MICROSOFT_LEARN_2026-09-11

7. Follow with me once

Use the normal Retail Orders fixture or a tiny named Fabric object. Before clicking anything, write the expected object/state and whether this will be LIVE FABRIC or OFFLINE PLAN. 1. Name dev, test and prod workspaces. 2. Map Git branch/review and deployment stage to those environments. 3. Move paths/connections/IDs into environment-specific configuration. 4. Write validation-after-deploy and rollback evidence before calling promotion safe. After each step, record the exact item/path/identity/run evidence you actually observed. If the portal or tenant is unavailable, stop the live claim, keep the design labeled OFFLINE, and use the official reference only to understand the object—not to fabricate a successful run.

```
Git branch/review -> DEV workspace -> TEST validation -> PROD promotion
logic: versioned
paths/connections/IDs: environment configuration
rollback: known reviewed version + validation
```

8. What you should see / be able to say

dev/test/prod mapping, Git branch/review path, environment configuration separation, validation-after-deploy and rollback plan.

9. Why it worked

The exercise separates the Fabric object from the durable state it controls, then attaches identity, validation and failure evidence. That makes the result explainable even when the portal layout changes.

DE15-15 - Solo proof, repair and mastery

10. Common beginner mistakes

- Copying portal steps without understanding the object/state created.
- Assuming an Admin experience equals a least-privilege user experience.
- Treating a successful run badge as enough without data reconciliation.
- Hard-coding workspace IDs, paths, credentials or environment-specific values.

11. If you get stuck

Preserve the first symptom. Check one boundary at a time: tenant/capacity -> identity/permission -> workspace/item -> path/grain/schema -> notebook/pipeline state -> serving -> monitoring/lifecycle. Compare current Microsoft Learn before changing multiple settings. If access is unavailable, stop and mark LIVE FABRIC pending.

12. You do a small version alone

Design a Git + deployment pipeline flow with dev/test/prod configuration separation and rollback evidence.

13. Validate

dev/test/prod mapping, Git branch/review path, environment configuration separation, validation-after-deploy and rollback plan. Label every proof LIVE FABRIC, OFFLINE PLAN or SIMULATOR. A green status alone is never sufficient.

14. Teach it back

Explain git, deployment and environment separation for 60–90 seconds. Name the object, the state it controls, one failure mode and one validation.

15. Protected review + changed retry

Save 04_PRACTICE_FILES/lesson_workspaces/DE15-15/attempt.md first. Only then open the protected Review. If weak, close Review and do this changed retry separately: Design a Git + deployment pipeline flow with dev/test/prod configuration separation and rollback evidence. Preserve the failed attempt append-only.

16. Evidence / mastery + exact next route

CREATE evidence/ only for genuine observed proof. Mastery requires the independent/changed proof, validation, teach-back, and no unresolved critical issue. Exact next route: DE15-16.

DE15-16 - End-to-end Fabric data engineering case

1. Why this matters

An end-to-end Fabric engineer must connect architecture, access, ingestion, lakehouse storage, Spark/notebook transformation, orchestration, SQL serving, monitoring and lifecycle management. The final proof is not "I clicked every menu." It is a reproducible system with evidence: what arrived, what was accepted/rejected, what changed incrementally, what ran, who could access it, and how you recovered from failure. In a job, a wrong assumption here can create data loss, excess privilege, unreproducible runs, or a cloud bill with no trustworthy output.

2. What you are learning

You will explain and apply End-to-end Fabric data engineering case, then produce this proof: end-to-end architecture/evidence index that distinguishes actual Fabric observations from offline design/simulator evidence.

3. Prerequisite / start state

DE15-15 mastered. Start with a blank lesson workspace; do not begin from the Review or a finished architecture plan.

4. Picture it in your head

Picture an evidence chain from source -> Bronze -> Silver -> Gold -> serving. Every arrow has identity, state, run evidence and recovery. The final skill is tracing backward from a wrong Gold result to the first broken boundary.

5. New words before we use them

lineage	Trace of how data moves/transforms between items.
evidence index	Map from claim/competency to inspectable proof.
failure drill	Intentional failure used to prove recovery behavior.
technical defense	Ability to explain design/state/evidence under questioning.

DE15-16 - Visual + follow with me once

6. Useful video / current source route

[VIDEO FIRST: Microsoft Learn official DP-700 video playlist](#)

Use: Use the full official DP-700 playlist as stage review; execute the course case rather than only watching | Status: PRESERVED_VISUAL_ROUTE_WITH_MICROSOFT_LEARN_FALLBACK_2026-09-11

[CURRENT OFFICIAL REFERENCE: DP-700T00-A end-to-end course](#)

Use: Current product behavior and exact lesson reference | Status: CURRENT_MICROSOFT_LEARN_2026-09-11

7. Follow with me once

Use the normal Retail Orders fixture or a tiny named Fabric object. Before clicking anything, write the expected object/state and whether this will be LIVE FABRIC or OFFLINE PLAN. 1. Start from the normal blank architecture plan, not the worked example. 2. Connect source -> Bronze -> Silver -> Gold -> serving, with notebook and pipeline responsibilities separated. 3. Add identity/security, monitoring and lifecycle controls. 4. Index the evidence for data controls, failure/recovery and every step actually executed vs only designed offline. After each step, record the exact item/path/identity/run evidence you actually observed. If the portal or tenant is unavailable, stop the live claim, keep the design labeled OFFLINE, and use the official reference only to understand the object—not to fabricate a successful run.

```
source -> Bronze -> Silver -> Gold -> SQL consumer
          ^ run IDs / counts / quarantine / versions / permissions / monitor
Wrong Gold -> trace backward to first mismatched boundary
```

8. What you should see / be able to say

end-to-end architecture/evidence index that distinguishes actual Fabric observations from offline design/simulator evidence.

9. Why it worked

The exercise separates the Fabric object from the durable state it controls, then attaches identity, validation and failure evidence. That makes the result explainable even when the portal layout changes.

DE15-16 - Solo proof, repair and mastery

10. Common beginner mistakes

- Copying portal steps without understanding the object/state created.
- Assuming an Admin experience equals a least-privilege user experience.
- Treating a successful run badge as enough without data reconciliation.
- Hard-coding workspace IDs, paths, credentials or environment-specific values.

11. If you get stuck

Preserve the first symptom. Check one boundary at a time: tenant/capacity -> identity/permission -> workspace/item -> path/grain/schema -> notebook/pipeline state -> serving -> monitoring/lifecycle. Compare current Microsoft Learn before changing multiple settings. If access is unavailable, stop and mark LIVE FABRIC pending.

12. You do a small version alone

Produce the complete normal-domain architecture/evidence plan from a blank template and explain it without the book open.

13. Validate

end-to-end architecture/evidence index that distinguishes actual Fabric observations from offline design/simulator evidence. Label every proof LIVE FABRIC, OFFLINE PLAN or SIMULATOR. A green status alone is never sufficient.

14. Teach it back

Explain end-to-end fabric data engineering case for 60–90 seconds. Name the object, the state it controls, one failure mode and one validation.

15. Protected review + changed retry

Save 04_PRACTICE_FILES/lesson_workspaces/DE15-16/attempt.md first. Only then open the protected Review. If weak, close Review and do this changed retry separately: Produce the complete clinic retest architecture/evidence plan from a blank template and explain it without the book open. Preserve the failed attempt append-only.

16. Evidence / mastery + exact next route

CREATE evidence/ only for genuine observed proof. Mastery requires the independent/changed proof, validation, teach-back, and no unresolved critical issue. Exact next route: G06 after M17.

M15 Mastery Checklist

Mark DEMONSTRATED only when your evidence supports the claim. LIVE FABRIC pending cannot be converted by confidence or percentage.

DE15-01	one architecture map that labels source, persistent state, compute/orchestration, consumer, identity bound	DEMONSTRATED / REPAIR / LIVE_PENDING
DE15-02	a permission matrix with at least one deliberately refused grant and one test performed as the intended n	DEMONSTRATED / REPAIR / LIVE_PENDING
DE15-03	three copy-vs-shortcut decisions with freshness, ownership and replay/recovery reasons.	DEMONSTRATED / REPAIR / LIVE_PENDING
DE15-04	a workspace/item inventory naming purpose, owner, input/output and validation evidence for every item.	DEMONSTRATED / REPAIR / LIVE_PENDING
DE15-05	explicit Files, Tables and shortcut paths plus a sentence explaining which bytes are copied and which are	DEMONSTRATED / REPAIR / LIVE_PENDING
DE15-06	table grain + schema contract + bad-row rules + 7 raw / 4 accepted / 3 quarantined / 3300 control for the	DEMONSTRATED / REPAIR / LIVE_PENDING
DE15-07	rerunnable notebook flow with parameters/config separated from transformation logic and a clean-start val	DEMONSTRATED / REPAIR / LIVE_PENDING
DE15-08	compatibility checklist that explicitly records local Spark 4.2/Delta 4.4 versus Fabric Runtime 2.0 Spark	DEMONSTRATED / REPAIR / LIVE_PENDING
DE15-09	dependency graph, parameter, safe-retry decision, blocking failure route and checkpoint/publish rule.	DEMONSTRATED / REPAIR / LIVE_PENDING
DE15-10	two read-only validation queries plus a justified lakehouse-vs-Warehouse choice for changed workloads.	DEMONSTRATED / REPAIR / LIVE_PENDING
DE15-11	Bronze/Silver/Gold contracts with layer grain, acceptance/rejection behavior, lineage and reconciliation.	DEMONSTRATED / REPAIR / LIVE_PENDING
DE15-12	base controls, update/insert controls, 5 rows / 4250 after merge, exact-rerun no-op and failed-publish ch	DEMONSTRATED / REPAIR / LIVE_PENDING
DE15-13	identity -> permission -> resource matrix, no literal secret, and evidence for one allowed plus one denie	DEMONSTRATED / REPAIR / LIVE_PENDING
DE15-14	monitoring record covering run status, freshness, quality/reconciliation, duration, failure detail and ca	DEMONSTRATED / REPAIR / LIVE_PENDING
DE15-15	dev/test/prod mapping, Git branch/review path, environment configuration separation, validation-after-dep	DEMONSTRATED / REPAIR / LIVE_PENDING
DE15-16	end-to-end architecture/evidence index that distinguishes actual Fabric observations from offline design/	DEMONSTRATED / REPAIR / LIVE_PENDING

M15 Revision Quick Reference

Current GA Spark runtime?	Runtime 1.3: Spark 3.5.5 / Python 3.11 / Java 11 / Delta 3.2.
Runtime 2.0?	GA (opt-in) Spark 4.1 / Python 3.13 / Java 21 / Delta 4.2.
Workspace role = data security?	No. Separate control-plane workspace/item permission from OneLake data-plane roles.
Copy or shortcut?	Choose from ownership, freshness, recovery/replay and security.
Pipeline vs Copy job?	Pipeline orchestrates dependent activities; Copy job is simplified data movement.
Lakehouse SQL endpoint writes?	No: read-only T-SQL over Delta tables; lakehouse data modification belongs to Spark/lakehouse path.
Succeeded = correct?	No: reconcile freshness, row/value controls and business state.
Git = deployment?	No: source versioning/integration and environment promotion are distinct controls.

M15 -> M16 Route

Correct next step

M15 mastery goes to M16 DataOps / CI-CD / Security / Observability. G06 remains locked until M15 + M16 + M17 are complete with required evidence.

Cloud runtime boundary

If you completed designs offline because Fabric access/capacity was unavailable, keep those claims pending. Do not invent tenant screenshots or run IDs. The later G06 gate must see genuine live evidence for cloud competencies it assesses.

Certification note

DP-700 is a useful Microsoft Fabric Data Engineer certification route, but course mastery is evidence-based and does not depend on buying or passing an exam.